

sources_1\new\SEG7_DISPLAY.vhd

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4
5
6  entity SEG7_DISPLAY is
7  Port(
8      reset: in std_logic;
9      clock: in std_logic;
10
11      score: in std_logic_vector(5 downto 0);
12
13      anodes: out std_logic_vector(3 downto 0);
14      seg7: out std_logic_vector(6 downto 0)
15  );
16  end SEG7_DISPLAY;
17
18  architecture SEG7_DISPLAY_ARCH of SEG7_DISPLAY is
19
20      constant ACTIVE: std_logic := '1';
21      constant DEFAULT: integer := 0;
22      constant COUNT_1KHZ: integer := (100000000/1000)-1;
23
24      constant SEG7_ZERO: std_logic_vector(6 downto 0) := "1000000";
25      constant SEG7_ONE: std_logic_vector(6 downto 0) := "1111001";
26      constant SEG7_TWO: std_logic_vector(6 downto 0) := "0100100";
27      constant SEG7_THREE: std_logic_vector(6 downto 0) := "0110000";
28      constant SEG7_FOUR: std_logic_vector(6 downto 0) := "0011001";
29      constant SEG7_FIVE: std_logic_vector(6 downto 0) := "0010010";
30      constant SEG7_BLANK: std_logic_vector(6 downto 0) := "1111111";
31
32
33      constant SELECT_DIGIT_0: std_logic_vector(3 downto 0) := "1110";
34      constant SELECT_DIGIT_3: std_logic_vector(3 downto 0) := "0111";
35      constant SELECT_NO_DIGIT: std_logic_vector(3 downto 0) := "1111";
36
37
38      signal selectedDigit: integer range 0 to 5;
```

```
39     signal p1_score:      integer range 0 to 5;
40     signal p2_score:      integer range 0 to 5;
41     signal enableCount:   std_logic;
42     signal digitSelect:   unsigned(1 downto 0);
43
44
45 begin
46     p1_score <= to_integer(unsigned(score(5 downto 3)));
47     p2_score <= to_integer(unsigned(score(2 downto 0)));
48
49     BINARY_TO_SEG7: seg7 <=  SEG7_ZERO  when selectedDigit = 0 else
50                             SEG7_ONE   when selectedDigit = 1 else
51                             SEG7_TWO   when selectedDigit = 2 else
52                             SEG7_THREE when selectedDigit = 3 else
53                             SEG7_FOUR  when selectedDigit = 4 else
54                             SEG7_FIVE  when selectedDigit = 5 else
55                             SEG7_BLANK;
56
57     DIGIT_SELECT: with digitSelect select
58                     selectedDigit <= p2_score when "00",
59                                     p1_score when "11",
60                                     DEFAULT when others;
61
62     ANODE_SELECT: process(digitSelect)
63     begin
64         case digitSelect is
65             when "00" => anodes <= SELECT_DIGIT_0;
66             when "11" => anodes <= SELECT_DIGIT_3;
67             when others => anodes <= SELECT_NO_DIGIT;
68         end case;
69     end process ANODE_SELECT;
70
71
72     SCAN_RATE: process(reset, clock)
73         variable count: integer range 0 to COUNT_1KHZ;
74     begin
75         if reset = ACTIVE then
76             count := 0;
77         elsif rising_edge(clock) then
78             if count >= COUNT_1KHZ then
79                 count := 0;
80             else
```

```
81         count := count + 1;
82     end if;
83 end if;
84
85 enableCount <= not ACTIVE;
86 if count = COUNT_1KHZ then
87     enableCount <= ACTIVE;
88 end if;
89 end process SCAN_RATE;
90
91 DIGIT_COUNT:process(reset, clock)
92 begin
93     if reset = ACTIVE then
94         digitSelect <= "00";
95     elsif rising_edge(clock) then
96         if enableCount = ACTIVE then
97             digitSelect <= digitSelect + 1;
98         end if;
99     end if;
100 end process DIGIT_COUNT;
101
102 end SEG7_DISPLAY_ARCH;
103
```